

Day04 课堂笔记

0. 复习反馈

多给一些作业以外的练习题

随机分配座位那 希望能在说一下 for循环那

希望可以多布置点作业,以便加强对知识点的进一步记忆!

讲的略快,下午可以带我们一起写xmind

如何分辨有些方法是有返回值的,有些是没有返回值得?

2. 字典

2.1 字典的定义和访问

```
1 # 字典 dict 定义使用{} 定义, 是由键值对组成(key-value)
2 # 变量 = {key1: value1, key2: value2, ...} 一个key:value 键值对是一个元素
3 # 字典的key 可以是 字符串类型和数字类型(int float), 不能是 列表
4 # value值可以是任何类型
5 # 1. 定义空字典
6 my_dict = {}
7 my_dict1 = dict()
8 print(my_dict, type(my_dict))
9 print(my_dict1, type(my_dict1))
10
11 # 2. 定义带数据的字典
12 my_dict2 = {'name': 'isaac', 'age': 18, 'like': ['学习', '购物', '游戏'], 1: [2, 5, 8]}
13 print(my_dict2)
14
15 # 3. 访问value 值, 在字典中没有下标的概念, 使用 key值访问对应的value 值
16 # 18
17 print(my_dict2['age'])
18 # '购物'
19 print(my_dict2['like'][1])
20
21 # 如果key值不存在
22 # print(my_dict2['gender']) # 代码报错, key值不存在
23 # 字典.get(key) 如果key值不存在, 不会报错, 返回的是None
24 print(my_dict2.get('gender'))
```

```

26 # my_dict2.get(key, 数据值) 如果key存在, 返回key对应的value值, 如果key不存在, 返回书写的数据值
27
28 print(my_dict2.get('gender', '男')) # 男
29 print(my_dict2.get('age', 1)) # 18
30
31 print(len(my_dict2)) # 4
32

```

2.2 字典中添加和修改数据

```

1 my_dict = {'name': 'isaac'}
2 print(my_dict)
3 # 字典中添加和修改数据, 使用key值进行添加和修改
4 # 字典[key] = 数据值; 如果key值存在, 就是修改, 如果key值不存在, 就是添加
5
6 my_dict['age'] = 18 # key值不存在, 添加
7 print(my_dict)
8
9 my_dict['age'] = 19 # key值已经存在, 就是修改数据
10 print(my_dict)
11
12 # 注意点 key 值 int 的 1 和 float 的 1.0 代表一个key值
13 my_dict[1] = 'int'
14 print(my_dict)
15 my_dict[1.0] = 'float'
16 print(my_dict)
17
18

```

```

{'name': 'isaac'}
{'name': 'isaac', 'age': 18}
{'name': 'isaac', 'age': 19}
{'name': 'isaac', 'age': 19, 1: 'int'}
{'name': 'isaac', 'age': 19, 1: 'float'}

```

2.3 字典中删除数据

```

1 my_dict = {'name': 'isaac', 'age': 19, 1: 'float', 2: 'aa'}
2
3 # 根据key值删除数据 del 字典名[key]
4 del my_dict[1]
5 print(my_dict)
6
7 # 字典.pop(key) 根据key值删除, 返回值是删除的key对应的value值
8 result = my_dict.pop('age')
9 print(my_dict)
10 print(result)
11
12 # 字典.clear() 清空字典, 删除所有的键值对
13 my_dict.clear()
14 print(my_dict)
15
16 # del 字典名 直接将这个字典删除了, 不能使用这个字典了
17 del my_dict # 后边的代码不能再直接使用这个变量了, 除非再次定义
18
19 # print(my_dict) 代码报错, 变量未定义
20

```

2.4 字典中遍历数据

1. for 循环直接遍历字典, 遍历的是字典的 key 值

```
1 my_dict = {'name': 'isaac', 'age': 18, 'gender': '男'}
2
3 # for 循环体直接遍历字典, 遍历的字典的key值
4 for key in my_dict:
5     print(key, my_dict[key])
6
```

2. 字典.keys()

```
7 # 字典.keys() 获取字典中所有的key值, 得到的类型是 dict_keys, 该类型具有的特点是
8 # 1. 可以使用list() 进行类型转换, 即将其转换为列表类型
9 # 2. 可以使用for循环进行遍历
10 result = my_dict.keys()
11 print(result, type(result))
12 for key in result:
13     print(key)
14
```

Run: 04-字典的遍历 x

```
C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day04/04-字典的遍历.py
dict_keys(['name', 'age', 'gender']) <class 'dict_keys'>
name
age
gender
Process finished with exit code 0
```

3. 字典.values()

```

16 # 字典.values() 获取所有的value值, 类型是 dict_values
17 # 1. 可以使用list() 进行类型转换, 即将其转换为列表类型
18 # 2. 可以使用for循环进行遍历
19 result = my_dict.values()
20 print(result, type(result))
21 for value in my_dict.values():
22     print(value)

```

for value in my_dict.values()

Run: 04-字典的遍历

```

C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day04/04-字典的遍历.py
dict_values(['isaac', 18, '男']) <class 'dict_values'>
isaac
18
男

```

4. 字典.items()

```

24 # 字典.items() 获取所有的键值对, 类型是 dict_items, key,value 组成元组类型
25 # 1. 可以使用list() 进行类型转换, 即将其转换为列表类型
26 # 2. 可以使用for循环进行遍历
27 result = my_dict.items()
28 print(result, type(result))
29 for item in my_dict.items():
30     print(item[0], item[1])
31
32 print('=' * 30)
33
34 for k, v in my_dict.items(): # k 是元组中的第一个数据, v 是元组中的第二个数据
35     print(k, v)
36

```

for k, v in my_dict.items()

Run: 04-字典的遍历

```

C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day04/04-字典的遍历.py
dict_items([('name', 'isaac'), ('age', 18), ('gender', '男')]) <class 'dict_items'>
name isaac
age 18
gender 男
=====
name isaac
age 18
gender 男

```

2.5 enumerate 函数


```

1 my_list = ['a', 'b', 'c', 'd', 'e']
2
3 # for i in my_list:
4 #     print(i)
5
6 for i in my_list:
7     print(my_list.index(i), i) # 下标, 数据值
8
9
10 # enumerate 将可迭代序列中元素所在的下标和具体元素数据组合在一块, 变成元组
11 for j in enumerate(my_list):
12     print(j)
13

```

```

0 a
1 b
2 c
3 d
4 e
(0, 'a')
(1, 'b')
(2, 'c')
(3, 'd')
(4, 'e')

```

2.6 公共方法

- **+** 支持字符串、列表、元组进行操作，得到一个新的容器
- ***** **整数** 复制，支持字符串、列表、元组进行操作，得到一个新的容器
- **in/not in** 判断存在或者是不存在，支持字符串、列表、元组、字典进行操作，注意：如果是字典的话，判断的是 key 值是否存在或不存在
- **max/min** 对于字典来说，比较的字典的 key 值的大小

```

Python Console>>> my_dict = {'a': 10, 'b': 20, 'c': 30}
>>> max(my_dict)           字典的 max, min, 比较的 key 值的大小
'c'
>>> min(my_dict)           a < z
'a'
>>> my_dict = {'a': 30, 'b': 20, 'c': 10}
>>> max(my_dict)
'c'
>>> min(my_dict)
'a'
>>> |

```

```
>>> 'A' < "Z"
True
>>> 'Z' < 'a'
True
```

总结答疑

1. 在字典中可以包含列表，列表中能包含字典吗？
- 2 可以的， 列表 元组可以存放任意类型的数据，同样，字典中的 value也可以是任意的类型，字典也可以作为字典的 value 值。

```
1 my_dict = {'name': 'isaac', 'age': 18, 'like': [1, 2], 'aa': {'a': 1, 'b': 200}}
2
3 print(my_dict)
4
5 print(my_dict['aa']['b'])
6
7 my_list = [{}, {}, {}, my_dict]
8 print(my_list)
9
```

Run: 06-答疑

C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day04/06-答疑.py

```
{'name': 'isaac', 'age': 18, 'like': [1, 2], 'aa': {'a': 1, 'b': 200}}
200
[{}, {}, {}, {'name': 'isaac', 'age': 18, 'like': [1, 2], 'aa': {'a': 1, 'b': 200}}]
```

Process finished with exit code 0

函数

- 1 `print()` 打印输出
- 2 `input()` 输入
- 3 `len()` 求容器长度的
- 4 ...
- 5 函数可以实现一个具体的功能

函数的定义和调用

```
1 # 函数,能够实现一个具体的功能,是多行代码的整合
2 # 函数的定义: 使用关键字 def ,
3 # def 函数名(): # 函数名要遵循标识符的规则, 见名知意
4 # 函数代码(函数体)
5 # 函数定义,函数中的代码不会执行,在函数调用的时候,才会执行
6 # 函数的好处: 重复的代码不需要多次书写, 减少代码冗余
7 print('函数定义前')
8
9
10 # 函数的定义, 函数的定义不会执行函数中算的代码
11 def func():
12     print('好好学习,天天向上')
13     print('good good study, day day up')
14     print('授课认真听讲,不要走神')
15
16
17 print('函数定义后')
18 # 函数调用的时候才会执行函数中的代码 函数名()
19 print('函数调用前')
20 func() # 代码会跳转到函数定义的地方去执行
21 print('函数调用后')
22
```

函数的文档说明

```

1  # 函数的文档说明本质就是注释, 告诉别人, 这个函数怎么使用的, 是干什么事的
2  # 只不过这个注释, 有特定的位置书写要求, 要写在函数名字的下方
3
4
5  def func():
6      """
7      打印输出一个hello world,
8      """
9      # aaa
10     print('hello world')
11
12
13     func()
14     # 查看函数的文档注释可以使用help(函数名)
15     # help(print)
16
17     help(func)

```

书写带参数的函数

好处： 可以使函数代码更加通用，适用更多的场景

```

1  # 定义一个函数, 实现两个数的和
2  def add(a, b): # a 和 b 称为是形式参数, 简称形参
3      # a = 10
4      # b = 20
5      c = a + b
6      print(f"求和的结果是{c}")
7
8
9  # 函数调用, 如果函数在定义的时候有形参, 那么在函数调用的时候, 必须传递参数值
10 # 这个参数称为 实际参数, 简称实参
11 # 在函数调用的时候, 会将实参的值传递给形参
12 add(1, 2)
13 add(100, 200)
14
15

```

函数定义时候的参数

函数调用时候的参数, 称为实参
会将实参的值给到形参

形参的个数需要和实参的个数对应, 相同, 不能多, 也不能少

Run: 09-带参数的函数 ×

C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day04/09-带参数的函数.py

求和的结果是3

求和的结果是300

局部变量

- 1 局部变量的作用域（作用范围）： 当前函数的内部
- 2 局部变量的生存周期： 在函数调用的时候被创建，函数调用结束之后，被销毁（删除）
- 3
- 4 局部变量只能在当前函数的内部使用，不能在函数的外部使用。

```
1  # 局部变量, 就是在函数内部定义的变量, 就是局部变量
2  # 局部变量, 只能在函数内部使用, 不能在函数外部和其他函数中使用
3
4
5  def func():
6      # 定义局部变量 num
7      num = 100
8      print(num)
9
10
11  def func1():
12      num = 200 # 这个num 和 func中的num 是没有关系的
13      print(num)
14
15
16  # 函数调用
17  func()
18  func1()
19
20  # 探究: 局部变量能否在函数外部使用
21  # print(num) # 代码报错, 局部变量不能在函数外部访问
22
```

全局变量

- 1 全局变量： 就是在函数外部定义的变量。
- 2 在函数内部可以访问全局变量的值，如果想要修改全局变量的值，需要使用 `global` 关键字声明

```
1 # 全局变量： 就是在函数外部定义的变量。
2
3 # 定义全局变量
4 g_num = 100
5
6
7 # 1. 能否在函数内部访问全局变量? ==> 可以直接访问全局变量的值
8 def func1():
9     print(g_num)
10
11
12 # 2. 能否在函数内部修改全局变量的值? ==> 不能直接修改全局变量的值
13 def func2():
14     # g_num = 200 # 这里不是修改全局变量的值, 是定义一个局部变量, 和全局变量的名字一样而已
15     # 想要在函数内部修改全局变量的值, 需要使用 global 关键字声明这个变量为全局变量
16     global g_num
17     g_num = 300
18
19
20 func1()
21 func2()
22 func1()
23
```

返回值

- 1 在函数中定义的局部变量，或者通过计算得出的局部变量，想要在函数外部访问和使用，此时就可以使用 `return` 关键字，将这个返回值返回

```

1  # 函数想要返回一个数据值, 给调用的地方, 需要使用关键字 return
2  # return 关键字的作用: ①, 将return 后边的数据值进行返回 ②, 程序代码遇到return, 会终止(结束)执行
3  # 注意点: return 关键字必须写在函数中
4
5
6  def add(a, b):
7      c = a + b
8      # 想要将求和的结果 c, 返回, 即函数外部使用求和的结果, 不在函数内部打印结果
9      return c
10 # 函数遇到return就结束了, 不会执行return之后的代码
11 print(f'求和的结果是{c}')
12
13 result = add(100, 200)
14 print(f'函数外部获得了求和的结果{result}')
15

```

Run: 12-函数的返回值 ×

C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day04/12-函数的返回值.py
函数外部获得了求和的结果300

Process finished with exit code 0

return 返回多个数据值

1. 程序代码遇到一个 return 之后, 后续的代码不再执行

```

1  def func(a, b):
2      c = a + b
3      d = a - b
4      # 需求: 想要将 c 和 d 都进行返回
5      # 思考: 容器可以保存多个数据值, 那就可以将 c 和 d 放到容器中进行返回
6      # return [c, d]
7      # return (c, d)
8      # return {'c': c, 'd': d}
9      # return {0: c, 1: d}
10     return c, d # 默认是组成元组进行返回的
11
12
13 result = func(10, 20)
14 print(f"a+b的结果是{result[0]}, a-b的结果是{result[1]}")
15

```

Run: 13-函数返回多个数据值 ×

C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day04/13-函数返回多个数据值.py
a+b的结果是30, a-b的结果是-10

Process finished with exit code 0

```

1  1. return 关键字后边可以不写数据值， 默认返回 None
2  def func():
3      xxx
4      return  # 返回 None, 终止函数的运行的
5
6  2. 函数可以不写 return, 返回值默认是 None
7
8  def func():
9      xxx
10     pass
11

```

函数的嵌套调用

```

1  def func1():
2      print('func1 start ... ')
3      print('函数的其他代码')
4      print('func1 end ...')
5
6
7  def func2():
8      print('func2 start ....')
9      func1() # 函数调用
10     print('func2 end....')
11
12
13 # 调用func1()
14 # func1()
15
16
17 # 调用func2()
18 func2()
19

```

在一个函数中可以调用另外一个函数

```

func2 start ....
func1 start ...
函数的其他代码
func1 end ...
func2 end....

```


